

Final Project Requirements

Purpose

Practice AI-assisted software engineering by solving algorithmic coding problems with two different AI coding agents. This is an individual assignment.

Introduction

The rapid emergence of AI-assisted coding tools has transformed the practice of software development. Where programmers once relied solely on their own expertise, intuition, and carefully curated libraries, they now have the ability to collaborate with intelligent coding agents that can propose solutions, generate boilerplate, optimize algorithms, and even explain design choices. This project invites you to step directly into that evolving landscape and critically evaluate how these tools influence problem solving, efficiency, and software design practices.

Rather than treating AI as a black box or shortcut, you will take on the role of an investigator: guiding two leading generative coding agents—Claude Sonnet 3.5 and GPT-4.1—through a series of algorithmic challenges. By systematically comparing their outputs in both Python and C++, you will not only test their correctness and efficiency but also reflect on the broader implications of using AI in professional software engineering.

This assignment is designed to replicate the kind of analytical work you might perform as a software engineer tasked with adopting new technologies in practice: identifying their strengths, diagnosing their weaknesses, and understanding how they fit into the broader toolkit of modern development. Through rigorous testing, time complexity analysis, and thoughtful reflection, you will gain first-hand insight into how AI can shape coding workflows—while also sharpening your ability to reason about algorithms, validate solutions, and present findings with professional clarity.

Objectives

1. Gain experience using AI-assisted coding agents for problem solving.
2. Compare quality, correctness, and performance of AI-generated solutions.
3. Evaluate the efficiency of AI coding assistants in terms of time-to-solution and algorithm complexity.

Project Requirements

You will primarily use the **python** programming language, and secondarily the C++ programming language (details below). I will configure the codespaces environment for this project to support both of these languages.

You will use GitHub Copilot within CodeSpaces to compare the following generative AI coding models:

- Claude Sonnet 3.5
- GPT 4.1

You will solve three algorithmic coding problems (one Easy, one Medium, one Hard) using both selected AI agents, in **python**. Each problem must be solved with both AI tools independently.

...continued on next page...

Selected LeetCode-Style Problems

Problem 1 (Easy): Two Sum

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Problem 2 (Medium): Group Anagrams

Given an array of strings `strs`, group the anagrams together.

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Problem 3 (Hard): Median of Two Sorted Arrays

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Input: `nums1 = [1,3]`, `nums2 = [2]`

Output: `2.0`

Explanation: merged array = `[1,2,3]` and median is `2`.

After you have used your two assigned AI tools to solve the 3 problems above (a total of 6 implementations), you will then solve just problem 3 using the C++ programming language, using both AIs, for a grand total of 8 implementations.

Identify your solutions by *Problem Number-AI Provider-AI Model-Language*:

Problem1-copilot-GPT-4.1-python

Problem1-copilot-Claude-Sonnet-3.5-python

Problem2-copilot-GPT-4.1-python

Problem2-copilot-Claude-Sonnet-3.5-python

Problem3-copilot-GPT-4.1-python

Problem3-copilot-Claude-Sonnet-3.5-python

Problem3-copilot-GPT-4.1-C++

Problem3-copilot-Claude-Sonnet-3.5-C++

Input/Output Files

For each problem, three sets of input/output test files are provided to evaluate correctness and time complexity scaling. Input sizes increase by a factor of 10 for each successive test case, to facilitate the time complexity analysis.

...continued on next page...

Final Report Requirements

Title Page

Include:

- Report Title
- Your Name
- Your GitHub ID
- Course: CSCI 630 (Online) *Semester (e.g. Fall 2025)*

Section 1: Executive Summary

Provide a summary of this assignment and the key take aways.

Section 2: Experimental Results

Present detailed results from solving each of the three algorithmic problems using the assigned AI tools and languages. For each problem, include:

- Time required to produce and debug a working solution (your total time spent, track accurately).
- Validation results against provided input/output test cases.
- A detailed analysis of the expected time complexity (Big-O) result based on the code generated.
- Analysis of the solution's actual time complexity (Big-O), including graphing the time spent on each data set, to show the time complexity trend.
- An explanation of any differences between the expected time complexity and the actual time complexity you measured.
- Observations about correctness, efficiency, and style of the solutions.

Tables and charts are required in this section. For tables, links to a google sheet are acceptable, just make you allow "Anyone with the link to view" permissions.

Section 3: Comparative Analysis of AI Tools

Compare the two AI tools systematically across all problems. Discuss:

- Speed of problem-solving.
- Correctness and reliability of solutions.
- Algorithmic efficiency of generated code.
- Ease of use, integration, and debugging experience.
- Strengths, weaknesses, and unique characteristics of each tool. Include a discussion of your experiences in each programming language.

Section 4: Lessons Learned

Reflect on your experience using AI-assisted coding tools. Discuss broader implications for:

- Software design, implementation and maintenance practices.
- Productivity and collaboration in software engineering.
- Potential risks and challenges associated with the use of AI for code generation.
- Benefits and limitations of AI assistance in professional settings.

Section 5: Conclusion and Future Directions

Summarize your findings. Suggest potential future applications of AI-assisted coding tools and outline how they might evolve to better support software design and maintenance.

References

All references must be cited in IEEE format. Include sources for AI tools, algorithm analyses, and any academic papers or documentation you reference.